

Corso di Laurea in Informatica A.A. 2023-2024

Laboratorio di Sistemi Operativi

Alessandra Rossi

Thread

- processi “leggeri”
- un processo può avere diversi thread
- i thread di uno stesso processo condividono la memoria ed altre risorse
 - facile comunicare tra thread!
- pthread = “POSIX thread”

Thread

I **thread** sono “**unità esecutive**” indipendenti all'interno di un processo, caratterizzate da:

- un **thread ID**;
- un **insieme di registri**, incluso un contatore di programma ed un puntatore di pila;
- una **pila** per le variabili locali e gli indirizzi di ritorno (di chiamate a funzioni);
- una **maschera dei segnali**;
- una **priorità**.

Tutti i thread relativi ad uno stesso processo ne condividono:

- lo **spazio di indirizzamento**;
- i **descrittori dei file** aperti;
- la **disposizione** ed i **gestori dei segnali**;
- le **credenziali**.

Thread

Consideriamo due processi che devono lavorare sugli stessi dati.

Come possono fare, se ogni processo ha la propria area dati (ossia, gli spazi di indirizzamento sono separati)?

- i dati possono essere scambiati mediante messaggi (pipe, FIFO) o tenuti in memoria condivisa

- i dati possono essere tenuti in un file che viene acceduto a turno dai due processi.

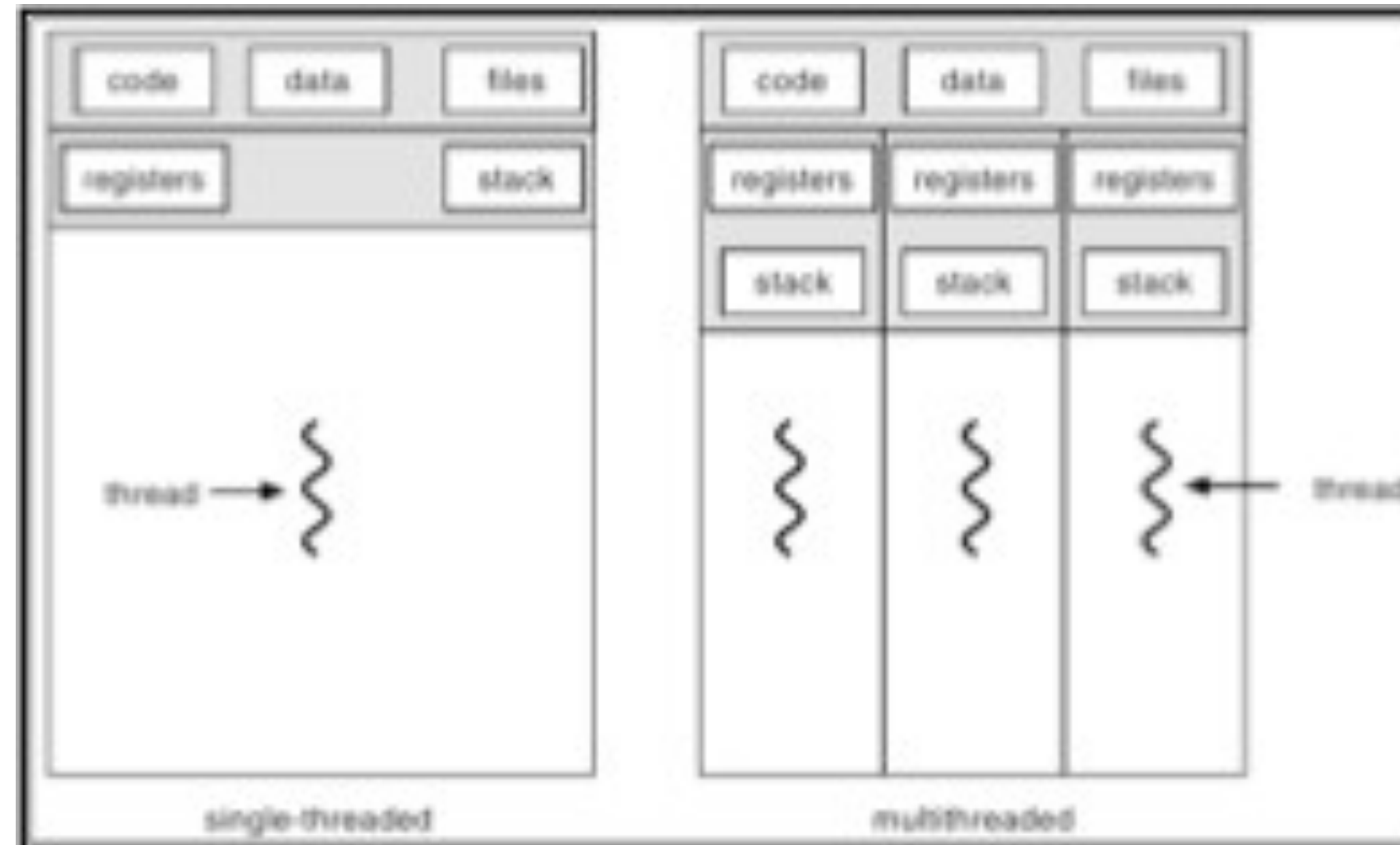
Non sarebbe comodo poter avere processi che possano automaticamente lavorare sugli stessi dati, senza usare meccanismi espliciti di condivisione/comunicazione?

Thread

- Il context switch tra processi richiede molto lavoro al SO: oltre a cambiare il valore dei vari registri, deve spostarsi dalle aree dati e di codice del processo uscente, a quelle del processo entrante.
- Se dati e codice di quest'ultimo erano stati swappati in memoria secondaria, occorre prima riportarli in memoria primaria.
- Se due (o più) processi potessero condividere dati e codice, il context switch fra di loro sarebbe molto più veloce
- Per soddisfare questo tipo di esigenza è nato il concetto di **thread**

Thread

Un processo **Multi-Thread** è fatto di più **thread**, detti peer thread.



Thread

- Ad ogni thread è associato in modo esclusivo il suo stato della computazione, fatto da:
 - valore del program counter e degli altri registri della CPU
 - uno stack
- Ma un thread condivide con i suoi peer thread:
 - il codice in esecuzione
 - i dati
 - i file aperti

Identificare i Thread

- processo ha `process id (pid)` di tipo `pid_t`
- thread ha `thread id (tid)` di tipo `pthread_t`

Nota: `pthread_t` struttura, funzione per il confronto:

```
int pthread_equal(pthread_t t1, pthread_t t2);
```

Ritorna non zero se uguali, 0 se diversi

```
pthread_t pthread_self(void);
```

Restituisce il tid del thread corrente

Creazione Thread

- Analogamente ai processi, quando un thread viene creato esso è associato ad un pezzo di codice;
- Tuttavia solo se il processo ospite (e quindi il suo corrispondente programma) è single-threaded, il thread è associato all'intero programma;
- Se il processo ospite è multi-threading, ciascun thread di tale processo è associato ad una funzione da eseguire, detta **funzione di avvio**;
- Anche i thread, come i processi, possono assumere gli stati **running, sleeping, blocked** e **terminated** ;
- Quando un thread termina la situazione è analoga al caso dei processi: è il programmatore che deve preoccuparsi affinché tutte le risorse impegnate dal thread siano rilasciate al sistema, facendo in modo che venga effettuata una **wait** a livello di thread.

Creazione Thread

- Quando un programma viene mandato in esecuzione tramite una chiamata **exec**, viene creato un singolo thread, detto **thread principale** o **iniziale**;
- Ulteriori thread vanno creati esplicitamente;
- Ogni thread ha numerosi **attributi**, da assegnare alla creazione: **livello di priorità**, **dimensione iniziale della pila**, etc;
- All'atto della creazione di un thread è necessario specificare la **funzione di avvio**: il thread inizierà la propria esecuzione richiamando la funzione di avvio;
- La **terminazione** di un thread può avvenire in tre diverse circostanze: **(a)** una chiamata esplicita di terminazione del thread, **(b)** la funzione di avvio ritorna, **(c)** il processo contenente il thread termina.

Funzione pthread_create

Per creare thread aggiuntivi relativi ad uno stesso processo, Posix prevede la funzione:

```
#include <pthread.h>

int pthread_create ( pthread_t *tid, const pthread_attr_t *attr,
                    void * ( *start_func) (void *), void *arg );
```

- se la chiamata ha successo, *tid* punta al thread ID;
- *attr* permette di specificare gli attributi del thread (se *attr* = NULL, gli attributi sono quelli di default);
- *start_func* è l'indirizzo della funzione di avvio;
- *arg* è l'indirizzo dell'argomento accettato dalla funzione di avvio;
- restituisce 0 in caso di successo, un intero positivo – secondo le convenzioni di <sys/errno.h> – in caso di errore.

Creare un thread

```
typedef void (*thread_start)(void *);  
  
int pthread_create(pthread_t *tid, const  
pthread_attr_t *attributes  
thread_start start,  
void *argument);
```

- Restituisce 0 se OK, un codice d'errore altrimenti
- tid = argomento di ritorno, conterrà il tid del nuovo thread
- attributes = attributi del thread (vedere dopo)
- start = indirizzo della funzione da cui partire
- argument = l'argomento passato alla funzione start

Trattare gli errori

- siccome i thread condividono la memoria, e' meglio non usare una variabile globale (come `errno`) per i codici d'errore
 - quindi, le funzioni pthread restituiscono direttamente un codice d'errore, e.g. `pthread_create`
-

`char *strerror(int n);`

- restituisce un messaggio corrispondente al codice d'errore `n`

Risorse condivise

- I thread di uno stesso processo condividono:
 - la memoria
 - il pid e il ppid
 - i file descriptor
 - le reazioni ai segnali
 - cioe', le chiamate a signal influenzano tutti i thread
- I thread non condividono: lo stack

Terminare un thread

- invocare `exit()` (`_exit`, `_Exit`) fa terminare *l'intero processo!*
- Analogamente un segnale ad un thread uccide il processo
- per terminare solo il thread corrente, si puo':
 - invocare return dalla routine di start, il valore di ritorno e' l'exit code
 - invocare `pthread_exit`
 - un altro thread del processo puo' chiamare `pthread_cancel`

Terminare un thread

Un thread può richiedere esplicitamente la propria terminazione grazie alla chiamata seguente, lasciando traccia del proprio stato di terminazione per quei thread che attendono per lui:

```
#include <pthread.h>

void pthread_exit ( void *status);
```

- *status* punta all'oggetto che definisce lo stato di terminazione del thread. Quest'ultimo **non** deve essere una variabile **locale** al thread chiamante, pena la sua scomparsa alla terminazione del thread stesso.

Aspettare la terminazione di un thread

Un thread può attendere per la terminazione di un altro thread relativo allo stesso processo:

```
#include <pthread.h>

int pthread_join ( pthread_t *tid, void **status );
```

- *tid* è l'ID del thread del quale si vuole attendere la terminazione;
- *status* punta al valore restituito dal thread per cui si è atteso, indicante il suo stato di terminazione (se *status* = NULL, tale stato non viene restituito);
- restituisce 0 in caso di successo, un intero positivo – secondo le convenzioni di `<sys/errno.h>` – in caso di errore.

Esempio: create, join

```
/* thread_create: stampa i TID del main thread e di due altri
thread */

#include <pthread.h>
#include <stdio.h>
#include <errno.h>

void *start_func(void *arg) /* funzione di avvio */
{
    printf("%s", (char *)arg);
    printf(" and my TID is: %d\n", (int)pthread_self());
}

int main(void)
{
    int en;
    pthread_t tid1, tid2;
    char *msg1 = "Hello world, I am thread #1";
    char *msg2 = "Hello world, I am thread #2";

    printf("The launching process has PID:%d\n", (int)getpid());

    printf("The main thread has TID:%d\n", (int)pthread_self());
```


Esempio: create, join

```
/* crea il 1mo thread */
if ((en = pthread_create(&tid1, NULL, start_func, msg1) != 0))
    errno=en, perror("pthread_create"), exit(1);

/* crea il 2ndo thread */
if ((en = pthread_create(&tid2, NULL, start_func, msg2) != 0))
    errno=en, perror("pthread_create"), exit(2);

/* attende per il 1mo */
if ((en = pthread_join(tid1, NULL) != 0))
    errno=en, perror("pthread_join"), exit(1);

/* attende per il 2ndo */
if ((en = pthread_join(tid2, NULL) != 0))
    errno=en, perror("pthread_join"), exit(2);

return 0;
}
```

Condivisione Memoria

- I due thread condividono lo stesso spazio di indirizzamento, e quindi vedono le stesse variabili: se uno dei due modifica una variabile, la modifica è vista anche dall'altro thread.

Variabili Globali

- Ma i thread di un task possono condividere variabili in maniera ancora più semplice, usando variabili globali.

```
#include <pthread.h>
#include <stdio.h>

int global_var = 5;

void *tbody(void *arg)
{
    printf(" ciao sono un thread, ora modifico una var globale\n");
    global_var = 27;
    *(int *)arg = 10;
    pthread_exit((int *)50); /* oppure return ((int *)50); */
}
```


Esempio: variabili globali, locali, allocazione dinamica

```
typedef struct foo{  
    int a;  
    int b;  
} myfoo;
```

```
myfoo test; // Variabile GLOBALE
```

```
void stampa(char *st, struct foo *test){  
    printf("%s: tid=%d a=%d b=%d\n", st, pthread_self(), test->a, test->b);  
}
```

```
void *fun1(void *arg){  
    myfoo test2 = {1,2}; // Variabile LOCALE  
    printf("%s %d\n", arg, pthread_self());  
    stampa(arg, &test2);  
    pthread_exit((void *)&test2);  
}
```


Esempio

```
void *fun2(void *arg){  
test.a = 3;  
test.b = 4; // Variabile GLOBALE  
printf("%s %d\n", arg, pthread_self());  
stampa(arg, &test); pthread_exit((void *)&test);  
}
```

```
void *fun3(void *arg){  
myfoo *test3;  
test3=malloc(sizeof(struct foo)); // Variabile allocata dinamicamente  
test3->a = 5;  
test3->b = 6;  
printf("%s %d\n", arg, pthread_self());  
stampa(arg, test3);  
pthread_exit((void *)test3); //c  
}
```

Cancellare un thread

```
int pthread_cancel(pthread_t tid);
```

- chiede che il thread specificato da tid venga terminato
 - non *aspetta* la terminazione
- restituisce 0 se OK, un codice d'errore altrimenti
- Come se pthread_exit() con il valore di uscita dato dalla costante PTHREAD_CANCELED

pthread_detach

In taluni casi è opportuno far sì che lo stato di terminazione di un thread T non venga memorizzato fintanto che un altro thread T' relativo allo stesso processo attenda per T , ma sia invece cancellato subito dopo la terminazione di T :

```
#include <pthread.h>

int pthread_detach ( pthread_t *tid);
```

- tid è l'ID del thread che si vuole distaccare;
- restituisce 0 in caso di successo, un intero positivo – secondo le convenzioni di `<sys/errno.h>` – in caso di errore.

però, poi non possiamo chiamare pthread_join

thread e fork

- Se un thread chiama fork, nasce un nuovo processo con un solo thread
- Potenziali problemi con i mutex in possesso di altri thread (vedere dopo)

G.S. -> per le exec invece...

“A call to any exec function from a process with more than one thread shall result in all threads being terminated and the new executable image being loaded and executed.
No destructor functions or cleanup handlers shall be called.”

<https://pubs.opengroup.org/onlinepubs/9699919799/functions/exec.html>

Thread e segnali

- Le chiamate a signal influenzano tutti i thread
- Se arriva un segnale a un processo, succede che:
 - se il processo ha impostato un handler, il segnale arriva ad *uno qualunque* dei thread (che esegue l'handler)
 - se invece la reazione al segnale consiste nel terminare il processo, *tutti i thread* vengono terminati

Inviare un segnale a un thread

```
int pthread_kill(pthread_t tid, int signo);
```

- manda il segnale signo al thread specificato da tid
 - se e' impostato un handler, viene eseguito nel thread tid
 - se non e' impostato un handler, e il comportamento di default e' di terminare il processo, vengono comunque terminati tutti i thread
- restituisce 0 se OK, un codice d'errore altrimenti

Attributi di un thread

- un thread può essere creato in “detached state” (stato sconnesso)
- un thread può bloccare i tentativi di essere cancellato (cancellabilità)
- altri attributi
 - posizione e dimensione dello stack
 - attributi real-time

Gestione Attributi

```
include <pthread.h>
```

```
int pthread_attr_init (pthread_attr_t *attr);
```

```
int pthread_attr_destroy(pthread_attr_t *attr);
```

- inizializza e distrugge una struttura per gli attributi di un thread. Uso:
 - si alloca una struttura pthread_attr_t (struttura opaca)
 - si chiama pthread_attr_init
 - si modificano gli attributi contenuti nella struttura usando apposite funzioni (vedere dopo)
 - si passa la struttura a pthread_create
 - si distrugge la struttura con pthread_attr_destroy
- restituiscono 0 se OK, un codice d'errore altrimenti

Detached State

- Se non ci interessa il valore di ritorno di un thread, conviene crearlo in *detached state*
 - però, poi non possiamo chiamare `pthread_join`

Impostare detached state

```
int pthread_attr_setdetachstate(pthread_attr_t *attr,  
                                int detachstate);
```

- imposta l'attributo detach-state della struttura puntata da attr
- l'argomento detachstate può essere:
 - PTHREAD_CREATE_JOINABLE (default)
 - PTHREAD_CREATE_DETACHED
- restituisce 0 se OK, un codice d'errore altrimenti

Cancellabilità

- In ogni istante, un thread può essere cancellabile o non cancellabile
- Quando partono tutti i thread sono cancellabili
- Quando un altro thread chiama `pthread_cancel`
 - se il thread è cancellabile, viene cancellato
 - se non è cancellabile, la richiesta di cancellazione viene memorizzata, in attesa che il thread diventi cancellabile

Impostare la Cancellabilità

```
int pthread_setcancelstate(int state, int *oldstate);
```

- imposta la cancellabilità a state e restituisce la vecchia cancellabilità in oldstate
- state e oldstate possono assumere i valori:
 - PTHREAD_CANCEL_ENABLE
 - PTHREAD_CANCEL_DISABLE
- restituisce 0 se OK, un codice d'errore altrimenti

Esercizio

- Creare un programma che prenda in input un numero e genera tanti thread quanti il numero indicato.
- Ogni thread può scrivere a terminare una stringa di benvenuto, numero del thread e tid
- Il processo principale attende la terminazione dei thread

Esempio: create, join

```
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int count = 0;

void *f(void *x)
{
    sleep(rand() % 10);
    count++;
    printf("Ciao! %d\n", count);
    return NULL;
}
```



```
int main(int argc, char *argv[])
{
    int i, err, n;

    if (argc != 2) {
        printf("Uso: %s <numero thread>\n", argv[0]);
        exit(1);
    }

    n = atoi(argv[1]);
    /* Allocazione consentita */
    pthread_t tid[n];

    for (i=0; i<n ;i++) {
        if ((err=pthread_create(&tid[i], NULL, f, NULL)) != 0)
            { printf("errore: %s\n", strerror(err));
              exit(1);
            }
    }

    for (i=0; i<n ;i++)
        pthread_join(tid[i],NULL);
    printf("finito.\n");

    return 0;
}
```

Esercizio

- Scrivere un programma che accetta un intero n da riga di comando, crea n thread e poi aspetta la loro terminazione
- Ciascun thread aspetta un numero di secondi casuale tra 1 e 10, poi incrementa una variabile globale intera ed infine ne stampa il valore
- Domanda: ci sono race conditions in questo programma?



Università degli Studi di Napoli Federico II

THANK YOU FOR YOUR ATTENTION

TRUST ME ...
I AM A ROBOT!

